DOCUMENTO TÉCNICO · WEB, SERVIDOR, BASE DE DATOS Y FIRMWARE

Recorrido del código y herramientas

Control de Inserción de Trama · 7mo Informática B, Instituto Leonardo Murialdo · 20/10/2026

Este documento cuenta cómo está armado el sistema por dentro: qué hace cada parte del código, dónde está, cómo viaja una orden desde el celular hasta el telar (y un aviso desde la botonera hasta la pantalla) y con qué herramientas se hizo. Sirve para orientarse antes de leer el código; el detalle de cada parte está en los README de cada carpeta y en la Documentación de Proyecto.

1. Las cuatro partes y cómo se hablan

El sistema tiene cuatro partes. La **página web** es lo que usan el operario y el cliente en el celular. El **servidor** (backend) guarda las reglas: quién puede hacer qué, en qué estado está el telar y por qué fila va el tejido. La **base de datos** guarda todo lo que tiene que durar. El **ESP32**, en el gabinete del telar, es el que toca la máquina. La web y el ESP32 nunca se hablan entre sí: los dos le hablan al servidor por HTTPS, y el servidor es el único que lee y escribe la base.

Parte	Dónde corre	Qué hace	Archivos principales
Página web	En el navegador; la sirve el mismo servidor	Ingreso, editor de dibujos, biblioteca, control del telar, ficha en PDF y vista del cliente	`backend/public/` `index.html`, `styles.css` y `vendor/`
Servidor	Render (Node.js 20 a 22)	API en `/api`, sesiones, permisos, estado del telar y cálculo de la posición	`backend/src/server.js`, `routes/`, `controllers/`, `middleware/`, `utils/`, `nivel2/`
Base de datos	Neon (PostgreSQL)	Dibujos, telares, producción, usuarios, errores y claves de los clientes	`backend/src/db/` `schema.sql` y las 24 migraciones
Firmware	ESP32 en el gabinete	Lee la botonera y el sensor, pulsa los relés y selecciona las bobinas	`esp32/` `control_trama_esp32/` (el `.ino` y cinco `.h`)

2. La página web (frontend)

Es una sola página, `index.html` (unas 4.600 líneas de HTML y JavaScript), con su hoja de estilos `styles.css`. No usa ningún framework: el JavaScript arma la pantalla directamente, lo que la hace liviana en un celular y fácil de servir. Tiene tres pantallas que se alternan sin recargar (**Inicio**, **Editor** y **Biblioteca**), y encima de ellas las capas de ingreso, recupero, la vista del cliente y los diálogos.

- **Ingreso.** Dos pestañas: «Personal» y «Clientes». El personal entra con la **huella** usando WebAuthn: el celular firma un desafío con su lector y el servidor verifica la firma; la huella nunca sale del teléfono. Si la huella no anda, se entra con el usuario y el código de recuperación. También se puede entrar como invitado, sin mover el telar.
- **Editor.** `renderGrid()` dibuja la grilla: cada columna es una bobina de selección y cada fila una pasada, con su número de pasadas seguidas (×) y la columna ⇄ para los intercalados. El dibujo viaja al servidor como datos JSON: la matriz, las pasadas de cada fila, los colores y los grupos intercalados. Cada cambio se guarda solo, en el momento; si todavía hay un guardado en camino, el siguiente espera a que termine.

- **Control del telar.** ▶️ manda `POST /api/telares/:id/asignar-patron` (un dibujo nuevo) o `/reanudar` (el mismo dibujo, donde quedó); ⏸️ manda `/pausar`, ⏪ `/retroceder` y ⏹️ `/detener`. Cada 4 segundos la página consulta el estado del telar y muestra lo que pasa en la máquina, también lo que alguien hizo desde la botonera o desde otro celular.
- **Ficha técnica en PDF.** Se arma en el mismo navegador con la librería **jsPDF**: el telar dibujado, la vista previa de la tela, la regla a escala, la producción y el hilado.
- **Vista del cliente.** Con su clave, el cliente ve solo su tejido; la página lo vuelve a pedir cada 15 segundos y dibuja la tela en un `canvas`.
- **Detalles comunes.** Modo oscuro, tamaños pensados para tocar con el dedo, textos para lectores de pantalla y tipografías IBM Plex Sans, IBM Plex Mono y Questrial (Google Fonts). Todos los pedidos al servidor pasan por `apiFetch()`, que manda la cookie de sesión y traduce los errores a mensajes claros.

3. El servidor (backend)

Está hecho en **Node.js con Express**. `server.js` arma la cadena por la que pasa cada pedido: cabeceras de seguridad (helmet), compresión, CORS, lectura del JSON, registro de pedidos (morgan) y un límite de pedidos por minuto (express-rate-limit). Después vienen las rutas, que llaman a los **controladores**, y al final un manejador de errores que siempre responde con un mensaje entendible. El mismo servidor entrega la página web. Al arrancar, `migrator.js` aplica las migraciones de la base que falten, cada una dentro de una transacción.

Rutas	Para qué
`/api/auth`	Sesión, registro y entrada con huella (@simplewebauthn/server), recupero con el código, modo invitado e invitaciones para registrar a alguien nuevo.
`/api/patrones`	Los dibujos: listar, crear, cambiar, borrar, metros por pasada, hilado, estadísticas y la clave del cliente.
`/api/telares`	El telar: estado, cargar un dibujo, pausar, reanudar, avanzar, retroceder, terminar, el control desde la página (solo sensado), los avisos de la botonera (`evento-fisico`) y la validación del conteo. Para el Nivel 2, `patron-actual` y `pasadas`.
`/api/historial`	La producción: cada tanda de tejido con sus fechas y sus pasadas.
`/api/errores`	Los avisos de error, también los que manda el ESP32.
`/api/cliente`	Entrar con la clave del tejido, ver el tejido y salir.

- **Quién es quién** (`middleware/auth.js`). El operario tiene una cookie de sesión firmada con HMAC-SHA256; el invitado, una sesión que no puede mover el telar; el ESP32 se identifica con la cabecera `X-Device-Key`; y el cliente tiene su propia cookie, que solo abre `/api/cliente`. Cada ruta dice cuál de ellos la puede usar.
- **Secretos.** Los códigos de recuperación y las claves de los clientes se guardan con un resumen SHA-256 (para encontrarlos cuando alguien los escribe) y cifrados con AES-256-GCM (para poder volver a mostrarlos). Las claves del servidor (`SESSION_SECRET`, `RECOVERY_SECRET`, `ESP32_DEVICE_KEY`) están en variables de entorno de Render, nunca en el repositorio.
- **La posición del tejido** (`utils/posicion.js`). Convierte las pasadas contadas en fila y pasada, con las repeticiones de cada fila y los intercalados. El firmware tiene el mismo cálculo (`posicion_dibujo.h`) y los dos se compararon en 2,4 millones de pasos.

- **Validación** (`utils/validacion.js`). De 1 a 300 filas y de 1 a 8 columnas, una sola bobina por fila, intercalados de dos filas o más y sin superponerse. Lo que no cumple se rechaza con un mensaje que dice qué corregir.

4. La base de datos

Es **PostgreSQL**, alojada en **Neon**. `schema.sql` crea las tablas una sola vez (`npm run init-db`) y las migraciones 001 a 024 la fueron cambiando a medida que se sumaron funciones; la tabla `migraciones_aplicadas` recuerda cuáles ya corrieron. `database/01_base_de_datos_completa.sql` junta todo en un solo archivo, generado por un script, para crear la base de una vez.

Tabla	Qué guarda
`patrones`	Los dibujos: la matriz, las pasadas de cada fila, los colores, los intercalados, los metros por pasada, el hilado y si lo hizo un invitado.
`telares`	Cada telar: su estado (tejiendo, pausado o detenido) y por qué, el dibujo cargado, el control desde la página y el último contacto del ESP32.
`historial_produccion`	Cada trabajo: cuándo empezó y terminó, la fila y la pasada donde va, las pasadas contadas y si el conteo ya se validó.
`usuarios`	El personal: nombre, usuario y el código de recuperación (resumido y cifrado).
`credenciales_biometricas`	La parte pública de cada huella registrada (WebAuthn). La huella en sí queda en el celular.
`desafios_webauthn`	Los desafíos de un solo uso para registrar una huella o entrar con ella.
`invitaciones`	Los códigos de invitación, de un solo uso, para registrar a alguien nuevo.
`errores_log`	Los errores y avisos de la web, del servidor y del ESP32.
`accesos_cliente`	La clave de cada tejido para su cliente, el nombre del cliente y los metros que encargó.
`migraciones_aplicadas`	Qué migraciones ya se aplicaron, y cuándo.

5. El firmware del ESP32

Está escrito en **C++ con Arduino** para el ESP32. Es un solo programa para los dos niveles: `NIVEL2_INSTALADO`, en `config.h`, decide si además de la botonera se usan el sensor de pasada y la selección de bobinas. Usa las librerías WiFi, HTTPClient y WiFiClientSecure (HTTPS), ArduinoJson para leer las respuestas, y FreeRTOS y el watchdog, que vienen con el ESP32.

- **Dos tareas.** La **tarea de red** consulta el estado del telar cada 2,5 segundos (`GET /api/telares/:id`), manda los avisos de la botonera (`POST evento-fisico`) y, en el Nivel 2, baja el dibujo (`GET patron-actual`) y reporta las pasadas cada segundo (`POST pasadas`). El **loop**, en el Nivel 2, lee el sensor de pasada y activa la bobina de la fila que toca.
- **Bloque A, la botonera.** Tres relés «aprietan» Marcha, Pausa y Retroceder durante 300 ms. El sensado de los tres botones entra por interrupciones, con antirrebote, sin confundir el pulso del propio relé con un botón y con un filtro para cuando se enciende el telar (los tres canales bajan juntos en menos de 150 ms y no se avisa nada).

- **Nivel 2.** `sensor_pasada.h` cuenta las pasadas, `seleccion_dibujo.h` activa los LCA110 de la fila y `posicion_dibujo.h` lleva la misma cuenta que el servidor, así sigue con la fila que toca aunque una consulta al servidor falle, y después le reporta la cuenta.
- **Seguridad.** Sin Wi-Fi no acciona nada: los relés quedan sueltos y la botonera sigue funcionando a mano. El watchdog reinicia la placa si algo se cuelga, los errores se guardan y se mandan cuando vuelve la red, y en solo sensado no pulsa ningún relé.

6. El recorrido de punta a punta

El operario toca ▶️ en el celular

1. La página manda `POST /api/telares/:id/asignar-patron` con el dibujo (o `/reanudar` si es el mismo dibujo), con la cookie de sesión.
2. El servidor verifica que sea un operario y que el control desde la página esté prendido. En una transacción, pone el telar en «tejiendo» y abre el trabajo en `historial_produccion`.
3. En la consulta siguiente (a lo sumo 2,5 segundos después), el ESP32 ve «tejiendo» y pulsa el relé de Marcha: la máquina arranca.
4. Con el sensor de pasada (Nivel 2), cada pasada suma: el ESP32 reporta la cuenta y el servidor calcula la fila y la pasada.
5. La página, que consulta cada 4 segundos, muestra «Tejiendo · Fila X / N» y marca la fila en la grilla.

Alguien aprieta Pausa en la botonera

1. La pausa le llega a la máquina directo, como siempre. El sensado del ESP32 la ve por una interrupción.
2. El ESP32 manda `POST /api/telares/:id/evento-fisico` con el tipo «pausa» y la cabecera `X-Device-Key`.
3. El servidor pone el telar en «pausado» y guarda que fue desde la botonera.
4. En su consulta siguiente, la página avisa «El telar se pausó desde la botonera o desde otra pantalla» y deja de avanzar.

Con el control desde la página apagado (**solo sensado**), el servidor responde a ▶️ cargando el dibujo pero sin arrancar, y rechaza ⏸️ y ⏪ con el código `CONTROL_WEB_DESACTIVADO`. La máquina se maneja con la botonera y la página sigue todo con el segundo recorrido.

7. Herramientas utilizadas

Herramienta	Para qué se usó
Lenguajes	JavaScript (la web y el servidor), HTML y CSS (la web), SQL (la base de datos), C++ (el firmware), Python (las simulaciones del firmware) y Markdown (la documentación técnica).
Node.js 20 a 22 y Express 4	El servidor. Librerías: pg (PostgreSQL), @simplewebauthn/server (huella), helmet, cors, compression, express-rate-limit, morgan y dotenv.
jsPDF y SimpleWebAuthn	En la página: la ficha técnica en PDF y la huella del lado del celular.
PostgreSQL en Neon	La base de datos en la nube.
Render	Donde se publica el servidor y, con él, la página web.

Herramienta	Para qué se usó
Arduino IDE 2 y el core ESP32	Escribir y cargar el firmware; ArduinoJson para el JSON.
GitHub	El repositorio, con ramas y pull requests para revisar cada cambio antes de pasarlo a la rama principal.
Three.js	El modelo 3D del gabinete, que se ve en el navegador (repositorio Modelado3D_CIT).
Pruebas	Pruebas propias en Node (`npm test` y una integración contra PostgreSQL real), Playwright con Chromium (la página en un navegador de verdad, con tamaño de celular), pruebas del firmware en la PC con g++ y simulaciones en Python, ESLint y html-validate.
Word, LibreOffice, Excel y Google Sheets	Los documentos y sus PDF, las planillas de componentes y verificaciones, y el Gantt.
Claude (Anthropic), con Claude Code	Inteligencia artificial usada como asistente de programación y de documentación: revisiones línea por línea del código y de los documentos, programación y pruebas de funciones (por ejemplo, solo sensado y la vista para los clientes), capturas para los manuales y redacción de documentos. Sus cambios quedaron en commits propios en GitHub, firmados como «Claude», y el equipo decide cuáles pasan a la rama principal.

Las pruebas en la planta, las mediciones del telar y el armado del hardware los hizo el equipo; lo que se probó por software y lo que falta probar con la máquina está en la Guía de mediciones y en el Checklist de verificaciones.